DATA-260 Lab 2 – Report

Pair 06 · Uday Patel (sections 1 to 6) and Sreeramachandra Sai Achutuni (sections 7 to 14)

Item	Value
Repository	https://github.com/UdayPate/data260-pair06 (private)
Tagged commit	_____ (to be filled in at the end: the commit tagged `lab2`; Lab 1 is tag `lab1`)
Environment	Windows with WSL 2, Docker Desktop (client and server 28.0.4), base image `python:3.12-slim`, MongoDB 7, Kafka 3.9.0 (KRaft), `confluent-kafka` 2.6.1, `pymongo` 4.10.1; local Kubernetes with `kind` 0.33.0 (Kubernetes 1.34.0, `kubectl` client 1.32.2)
Local model	`qwen3:8b` via Ollama (partner sections)

This report is written in two parts. Sections 1 to 6 are Uday's: Docker, Docker Compose, Kubernetes manifests, Kafka and the MongoDB migration. Sections 7 to 14 belong to the partner and are marked with red dashed boxes. Every claim in sections 1 to 6 comes from a file in the repository: `RUN_LOG.txt` (real commands and output, failures included), `docs/lab2/kafka_demos_output.txt`, `docs/lab2/`, `k8s/`, and the three screenshots in `docs/lab2/screenshots/`. The list is at the end of section 6.

1. Pair parameters and seed counts

Parameter	Rule	Value
PAIR	assigned on Canvas	**06**
PORT_BASE	9000 + (PAIR × 10)	**9060**
Gateway (nginx, Docker and Kubernetes) / single app for local development	PORT_BASE	**9060**
Website	PORT_BASE + 1	**9061**
Student / Company / Job / Application / Event / AI Assistant services	PORT_BASE + 2 to + 7	**9062 / 9063 / 9064 / 9065 / 9066 / 9067**
Kafka (from this machine) / MongoDB (from this machine)		9068 / 9069
Database, images, containers, topics, namespace	prefix `p{PAIR}`	database `p06_handshake`, images `p06-<service>:1.0.0`, topics `p06.<name>` (dead letter `p06.dlq`), namespace `p06-handshake`
SEED	PAIR, for every generator	**6** (`random.Random(6)`, `Faker.seed(6)`)
CITY_SET	assigned on Canvas	**San Jose, Sunnyvale, Mountain View**

The seed script (`backend/seed/generate_seed.py`) was rewritten for MongoDB but keeps the order of the random numbers, so the counts are exactly the Lab 1 counts. It was run three times with the same result: on the first MongoDB, in the Docker Compose stack and in the Kubernetes cluster (`RUN_LOG.txt` 2.2, 3.5, 6.4).

Collection	Documents	Lab 1 minimum
companies	50	50
students	200	200
jobs	300	300
applications	1,500	1,500
events	100	100
event_registrations	400	
student_preferences	151	
saved_items	150	
student skills (inside the student documents)	877	
job skills (inside the job documents)	1,073	

The seeded demo logins are `demo.student@sjsu.edu` and `demo.company@example.com`.

2. Architecture

The platform from Lab 1 was not rebuilt. It was split into six services that share one codebase: `backend/app/` holds the routers, schemas and services, and `backend/services/<name>/main.py` mounts only the routers of that service. A test (`tests/test_services.py`) checks that the six services together serve exactly the endpoints of the single app, with none missing and none served twice. Today that is 42 endpoints: the 40 from Lab 1, `GET /media/profile_pics/{name}` (pictures now come from MongoDB) and `PUT /applications/{id}/resume`.

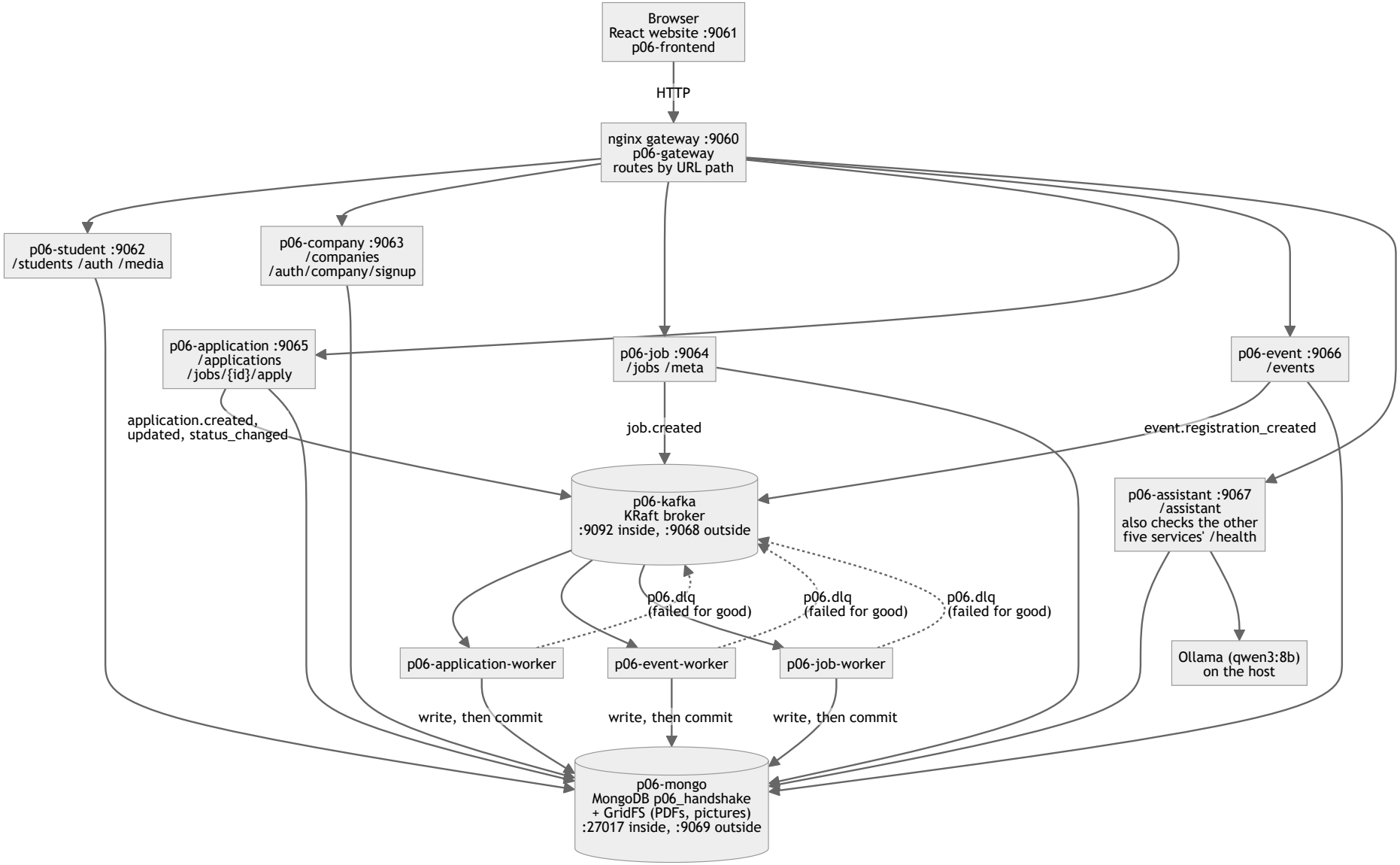


Figure 1: The containers of the stack and how they talk. Names and ports are the Docker Compose ones; the Kubernetes names are the same.

The browser only knows one address, the nginx gateway on 9060. The gateway sends each URL path to the service that owns it (`gateway/nginx.conf`); `/jobs/{id}/apply` and `/jobs/{id}/applications` go to the Application service and the rest of `/jobs` to the Job service. All services use the same MongoDB database and the same JWT secret. The services that publish events (Application, Event, Job) talk to Kafka, and three worker containers consume the events and write to MongoDB.

Design decisions that shape the rest of this report:

Decision	Why
One codebase, six entry points	Lab 2 asks for six services but not a rewrite; the Lab 1 tests keep working.
nginx gateway routes by path	The website keeps calling one address. The names in `nginx.conf` (`p06-student`, `p06-job`, ...) are the Docker Compose service names and the Kubernetes Service names, so one file works in both.
Resumes and pictures are stored in MongoDB (GridFS)	Every container is stateless: no shared volume, any replica can serve any file.
Applying is asynchronous	`POST /jobs/{id}/apply` stores the PDF, publishes `application.created` and answers 202 Accepted; the Application worker saves the application.
Integer ids are kept (a `counters` collection)	URLs such as `/jobs/12`, the website and the assistant tools do not change.
The assistant's tools stay inside the assistant service	They reuse the service functions of the shared data layer, so they follow the same permission rules as the API.

Component	Port	Image	Role
p06-student	9062	p06-student:1.0.0	accounts, login, student profiles and pictures
p06-company	9063	p06-company:1.0.0	company accounts and profiles
p06-job	9064	p06-job:1.0.0	job postings and search; publishes `job.created`
p06-application	9065	p06-application:1.0.0	applying and applications; publishes `application.*` events
p06-event	9066	p06-event:1.0.0	events and registrations; publishes `event.registration_created`
p06-assistant	9067	p06-assistant:1.0.0	the AI assistant; calls Ollama on the host
p06-application-worker, p06-event-worker, p06-job-worker	none	same images as the services	Kafka consumers
p06-gateway	9060	p06-gateway:1.0.0	nginx, routes by path
p06-frontend	9061	p06-frontend:1.0.0	the React website, served by nginx
p06-mongo	27017	mongo:7	MongoDB with GridFS
p06-kafka	9092	apache/kafka:3.9.0	one broker, KRaft mode (no ZooKeeper)

The apply request shows how the pieces work together. The 202 answer comes back as soon as the broker has confirmed the message; the worker saves the application a moment later and commits its Kafka offset only after the database write. The website's apply box shows "received" and asks again every 1.5 seconds, at most 10 times, until the application appears.

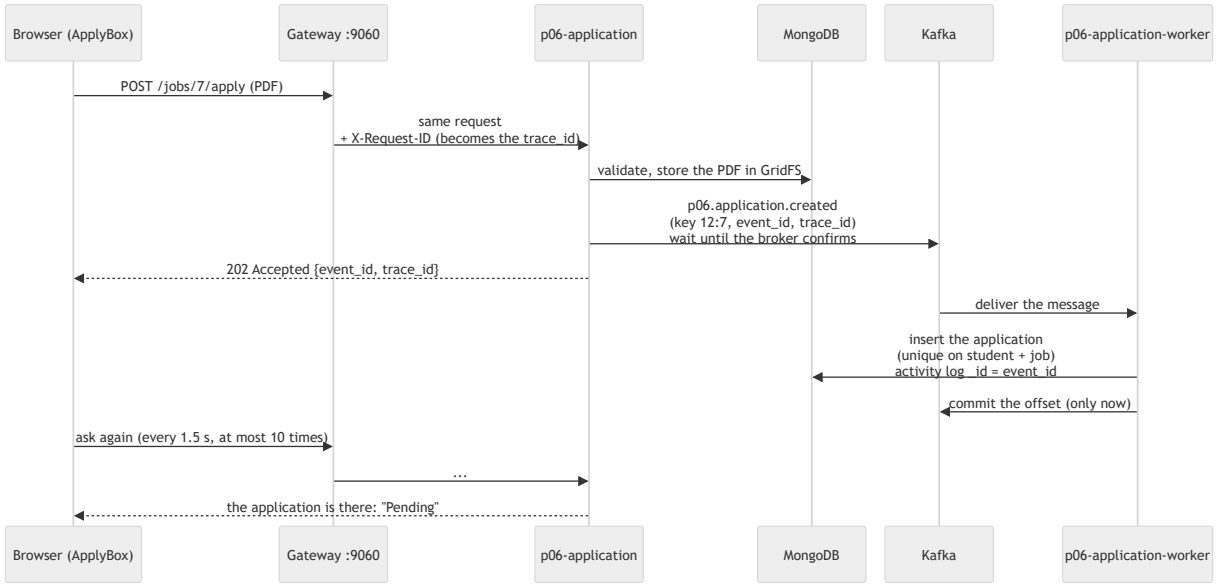


Figure 2: The apply flow. The same trace_id (the gateway's X-Request-ID) is in the API log, the Kafka headers and body, the worker log and the saved documents.

3. Docker and Docker Compose

3.1 The images

There is one Dockerfile per service in `backend/docker/` (`Dockerfile.student`, `.company`, `.job`, `.application`, `.event`, `.assistant`), one for the gateway and one for the website. The three Kafka workers use the images of their service with a different command (`python -m services.<name>.worker`). Each backend image installs only its own requirements (`backend/requirements/<service>.txt`, which include `base.txt`), so for example the Job image has no `python-multipart` and no `httpx`, and only the images that talk to Kafka carry `confluent-kafka`.

```
FROM python:3.12-slim
WORKDIR /srv
COPY requirements/base.txt requirements/application.txt requirements/
RUN pip install -r requirements/application.txt           # before the code, so this layer is reused
COPY app/ app/
COPY services/__init__.py services/common.py services/
COPY services/application/ services/application/
RUN useradd --system --uid 10001 p06
USER p06                                                  # not root
EXPOSE 9065
HEALTHCHECK --interval=15s --timeout=3s --start-period=15s --retries=3 \
    CMD python -c "... urlopen('http://127.0.0.1:9065/health') ..."
CMD ["uvicorn", "services.application.main:app", "--host", "0.0.0.0", "--port", "9065"]
```

Image	Port	Packages beyond the shared base	Size
p06-student	9062	python-multipart	252 MB
p06-company	9063	python-multipart	252 MB
p06-job	9064	confluent-kafka	267 MB
p06-application	9065	python-multipart, confluent-kafka	267 MB
p06-event	9066	confluent-kafka	267 MB
p06-assistant	9067	httpx	254 MB
p06-gateway	9060	nginx-unprivileged	73.7 MB
p06-frontend	9061	built with Node 20, served by nginx-unprivileged	74.9 MB

(Sizes from `docker images "p06-*"` on 2026-11-09. The shared base is FastAPI, uvicorn, pymongo, Pydantic, bcrypt and PyJWT.)

What was checked inside the running containers (`RUN_LOG.txt` 1.3 to 1.8, 2.6):

- All eight images built with exit code 0. The first frontend build failed with a network timeout contacting Docker Hub, not with a Dockerfile error; after the base images were pulled again it built.
- The services run as user `p06`, the website and gateway as `nginx`, none as root. No `.env` file, no tests folder and no SQL or test libraries are inside the images.
- Every service answers `GET /health` with `{"status":"ok","service":"p06-<name>","pair":"06","port":...}`.
- The Dockerfile's health-check command really can fail: run inside `p06-event` it exits 1 against a wrong port and 0 against port 9066.

3.2 Health checks

Check	What it tests	Used for
`GET /health`	the process is alive; it does not touch the database	the image `HEALTHCHECK`; Kubernetes liveness and startup probe
`GET /health/ready`	MongoDB answers a ping; HTTP 503 if not	Kubernetes readiness probe
`GET /health/dependencies` (assistant only)	MongoDB, the five other services, Kafka and Ollama	the Compose check that the assistant reaches its dependencies (3.4)
MongoDB container	`mongosh` ping, every 10 s	Compose `depends_on: service_healthy`
Kafka container	`kafka-broker-api-versions.sh`, every 15 s	Compose `depends_on: service_healthy`
Worker containers	a heartbeat file touched every second must be less than 30 s old	Compose and Kubernetes (workers have no web address)

The split between `/health` and `/health/ready` was tested by stopping MongoDB while the services kept running (`RUN_LOG.txt` 3.9). `/health` stayed 200, `/health/ready` answered 503 with `"mongodb":"unreachable"`, a login through the gateway answered 500 `A database error occurred`, and the containers stayed `(healthy)` because the Docker check does not need the database. After MongoDB was started again, `/health/ready` was 200 and login worked without restarting any service. Finding this also exposed a real weakness: a service that

starts before MongoDB (normal in Kubernetes) never created its indexes, including the unique rules. `get_db()` now creates the indexes on the first request that finds MongoDB, and a test covers it.

3.3 Docker Compose

`docker-compose.yml` (project name `p06-handshake`) defines 13 containers on one network, `p06-net`, and two named volumes, `p06-mongo-data` and `p06-kafka-data`: MongoDB, Kafka, the six services, the three workers, the gateway and the website. Ports are published on `127.0.0.1` only, except the gateway (9060) and the website (9061). Startup order comes from `depends_on` with `condition: service_healthy`: MongoDB and Kafka first, then the services, then the assistant, then the gateway and the website. The file contains no password: it uses `${MONGO_PASSWORD:?...}` and `${JWT_SECRET:?...}` references that Docker Compose reads from the git-ignored root `.env` (`.env.example` holds placeholders), and Compose refuses to start with a clear message if a value is missing.

```
docker compose build          # the eight images, p06-<name>:1.0.0
docker compose up -d          # about 25 to 35 s until everything is (healthy)
docker compose ps
docker compose down           # the data volumes stay;  down -v also deletes the data
```

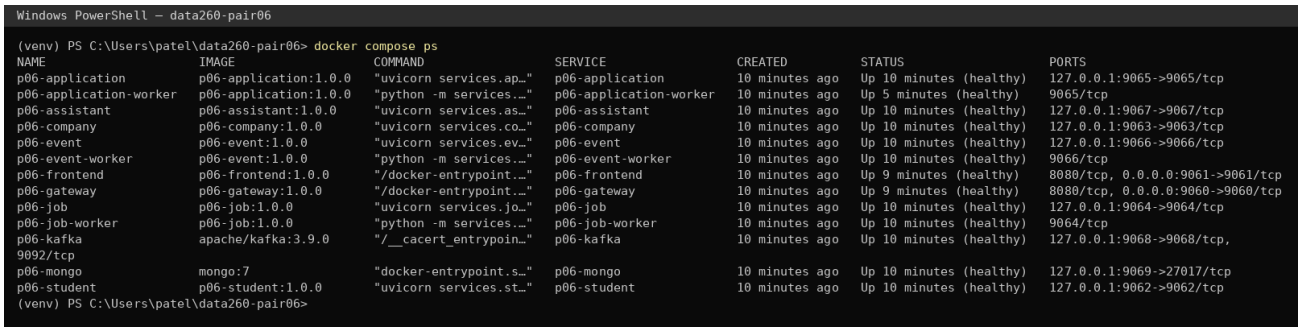


Figure 3: docker compose ps: all 13 containers are healthy, with the p06- names, images and ports.

3.4 How the parts reach each other

Website to backend. The browser calls the gateway at `http://localhost:9060`. That address is the build argument `VITE_API_URL` of the website image, which Vite writes into the JavaScript bundle; `grep` finds it once in the built files. A browser check (CORS preflight from `http://localhost:9061` to `GET /jobs` with an `Authorization` header) answered 200 with `access-control-allow-origin: http://localhost:9061` and `x-p06-service: p06-job`. Through the gateway, 21 sample requests reached the right service (for example `/students/me` → p06-student, `/jobs/12/apply` → p06-application, `/events` → p06-event, `/assistant/chat` → p06-assistant; `RUN_LOG.txt` 1.6).

Backend to MongoDB. Every service gets one connection string, `MONGO_URL=mongodb://<user>:<password>@p06-mongo:27017/?authSource=admin`, from its environment; `p06-mongo` is the container name on `p06-net`. Because files are in GridFS and sessions are in MongoDB, the containers are stateless. This was tested across containers: a company logo uploaded through the Company container was served by the Student container with the same bytes, and a login session created by the Student service was accepted by the Application service (6 of 6 checks passed). The 67 live checks that cover counts, indexes, bcrypt login, sessions, GridFS files, role privacy and the main workflows were run against the Compose stack with a password on MongoDB: 67 passed, 0 failed (`RUN_LOG.txt` 3.8).

Assistant to the other services. The assistant container asks for MongoDB, the five other services (by name and port, for example `http://p06-job:9064/health`), Kafka and Ollama. Ollama runs on the laptop, not in

Docker; the container reaches it at `http://host.docker.internal:11434`.



Figure 4: GET /health/dependencies on the assistant service: MongoDB and the five services are "ok" (required); Kafka and Ollama are "ok" (optional).

The assistant also worked end to end through the stack (browser address → gateway → assistant container → Ollama → tools → MongoDB; `RUN_LOG.txt` 3.7). "Find a remote data science internship" called `search_jobs` and returned three jobs. "Please save job 1" asked for confirmation; answering "no" left `saved_items` at 150, and answering "yes" saved it (151) and wrote an `item_saved` activity-log entry.

3.5 Tests

The backend suite grew from the 405 tests of Lab 1 to 543 tests, all passing (run on 2026-11-09 by Uday: `543 passed in 345.60s`). The new tests cover the service split, the health endpoints, the MongoDB data layer and the Kafka code. A mutation check showed that the split test can fail: removing `/auth/me` from the Student service made it fail, and restoring the file made it pass again.

4. Kubernetes manifests

The manifests are in `k8s/`. Everything lives in the namespace **`p06-handshake`** and every resource name starts with `p06-`. They were tested on a local `kind` cluster; the AWS (EKS) deployment is the partner's section 7.

```
k8s/base/                one file per component + kustomization.yaml (26 objects)
k8s/overlays/local/      the kind version: NodePort Services for the gateway (30060) and the website (30061)
k8s/kind-config.yaml     the local cluster, maps localhost:9060 and :9061 into it
k8s/create_secret.py     creates the Secret p06-secrets from the git-ignored .env
k8s/load_images.py       copies the local Docker images into the kind cluster
k8s/secret.example.yaml  a template with placeholders only
```

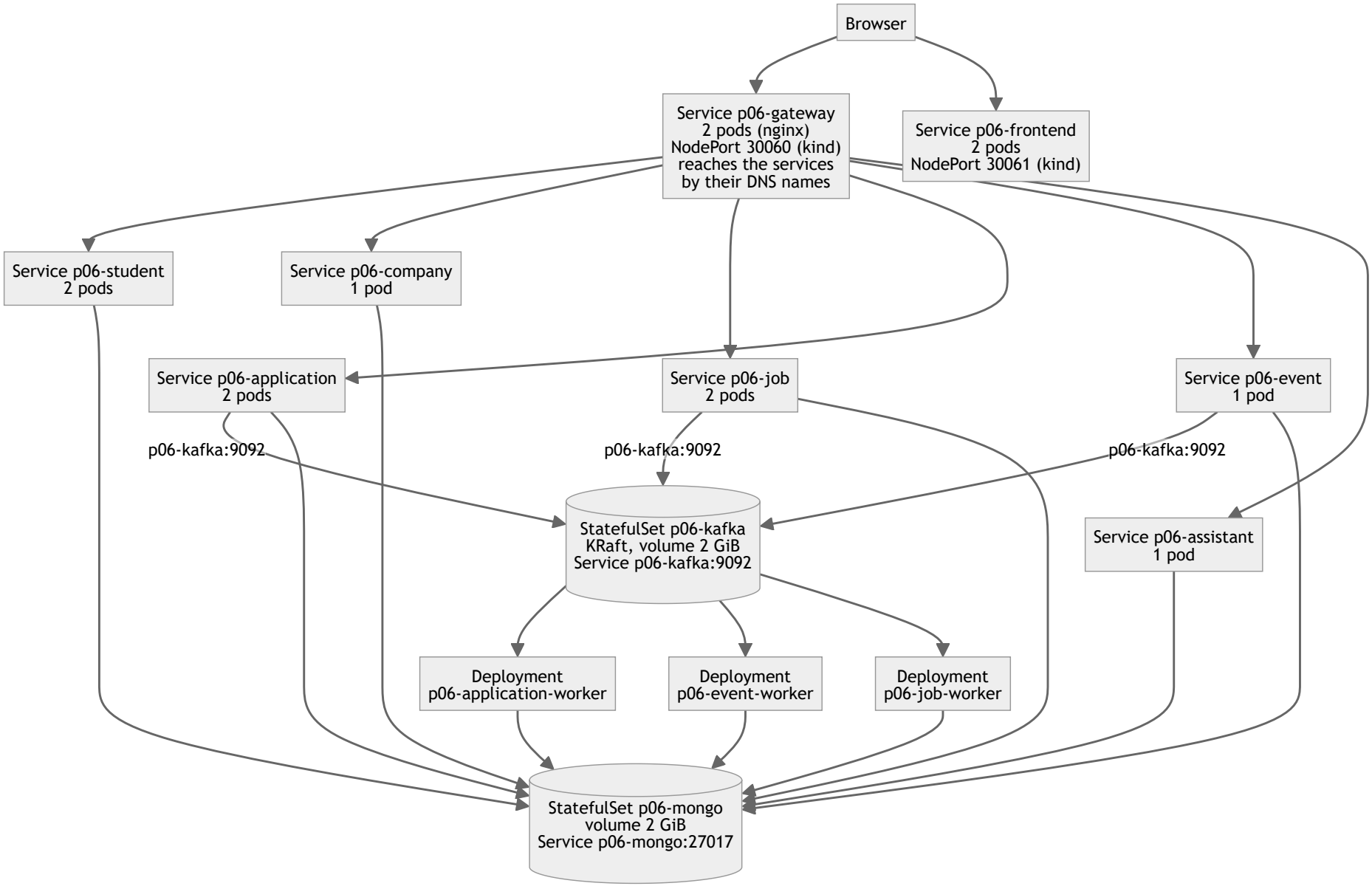


Figure 5: The Kubernetes version of the system. The ConfigMap p06-config and the Secret p06-secrets are read by every service and worker pod as environment variables (not drawn).

4.1 What is deployed

Workload	Kind	Replicas	CPU request / limit	Memory request / limit
p06-student, p06-job, p06-application	Deployment + Service	2 each	50m / 500m	96Mi / 384Mi
p06-company, p06-event, p06-assistant	Deployment + Service	1 each	50m / 500m	96Mi / 384Mi
p06-application-worker, p06-event-worker, p06-job-worker	Deployment (no Service)	1 each	50m / 500m	96Mi / 384Mi
p06-gateway, p06-frontend	Deployment + Service	2 each	25m / 250m	32Mi / 128Mi
p06-mongo	StatefulSet + Service, 2 GiB volume	1	100m / 1 CPU	256Mi / 768Mi
p06-kafka	StatefulSet + Service, 2 GiB volume	1	200m / 1 CPU	512Mi / 1Gi

That is 18 pods. The workers have no Service because nothing calls them; they read from Kafka. The Kafka manifest runs one broker in KRaft mode with automatic topic creation switched off, so a misspelled topic name fails instead of silently creating a new topic.

4.2 ConfigMaps and Secrets

Nothing secret is committed. The ConfigMap `p06-config` holds the database name, the allowed website origins (`CORS_ORIGINS`), the Kafka address (`p06-kafka:9092`) and the Ollama address. The Secret `p06-secrets` holds `MONGO_USER`, `MONGO_PASSWORD` and `JWT_SECRET`; it is not in `kustomization.yaml`. `k8s/create_secret.py` reads those three values from the git-ignored `.env`, passes them to `kubectl` through a temporary file that only the owner can read and deletes it afterwards, so the values are never printed or put on a command line. The pods build the connection string themselves from the Secret: `mongodb://$(MONGO_USER):$(MONGO_PASSWORD)@p06-mongo:27017/?authSource=admin`. A search of `k8s/` for the real password and the real JWT secret found no match.

4.3 Service discovery, probes and scaling

Every Service name is a DNS name inside the namespace. The gateway's `proxy_pass http://p06-job:9064` is resolved by Kubernetes to the Service, which spreads the requests over the ready pods behind it; the services reach MongoDB at `p06-mongo:27017` and Kafka at `p06-kafka:9092` the same way. The assistant's `/health/dependencies` run inside its pod reported MongoDB and all five services `ok`, which shows the names resolve.

Each service pod has a startup probe and a liveness probe on `/health` and a readiness probe on `/health/ready`: a failed liveness probe restarts the pod, a failed readiness probe removes it from the Service. A worker's liveness probe checks the heartbeat file. There is no start-order setting: a pod that starts before MongoDB or Kafka is simply not ready (or retries) until they are up. Replica counts and resource requests and limits are in the manifests so the scheduler knows what fits. To scale by hand: `kubectl -n p06-handshake scale deployment/p06-job --replicas=4`. No autoscaler is configured.

4.4 Test on kind

(`RUN_LOG.txt` section 6; the cluster `p06-handshake` was created from `k8s/kind-config.yaml`.)

Step	Result
Cluster and images	cluster ready in 31 s; 10 images loaded (8 `p06-*`, `mongo:7`, `apache/kafka:3.9.0`) in 1 min 9 s
`kubectl apply -k k8s/overlays/local`	all 18 pods `1/1 Running` after 60 s, 0 restarts; both volume claims `Bound`
Seed the cluster's MongoDB	the same counts as in section 1
Gateway and website	`GET localhost:9060/health` → `{"status":"ok","service":"p06-gateway","pair":"06"}`; website HTTP 200
Login, job search, apply	login returns a token; `GET /jobs` total 121; `POST /jobs/294/apply` → 202 with `event_id` and `trace_id`; the Application worker pod logged `event=processed` with the same ids; `GET /applications/mine` answered with a total of 12
Assistant dependencies	MongoDB, student, company, job, application, event `ok`; Kafka `ok`; Ollama `ok`
Kafka topics created inside the cluster	`p06.application.created`, `.updated`, `.status_changed`, `p06.event.registration_created`, `p06.job.created`, `p06.dlq`
Delete a `p06-job` pod during 20 authenticated requests (one every 0.5 s)	20 of 20 answered 200; a replacement pod was running 25 s later
Scale `p06-job` 2 → 4 → 2	4/4 ready after about 25 s, the Service listed 4 endpoints
Restart the MongoDB pod	login works afterwards and the demo student still has 12 applications: the data is on the volume

```
NAME                                    READY   STATUS    RESTARTS   AGE
p06-application-67f8d95457-mjz5f        1/1     Running   0          68s
p06-application-67f8d95457-rsn9l        1/1     Running   0          68s
p06-application-worker-565d45f678-czn2k 1/1     Running   0          68s
p06-assistant-5484464d5f-54c9q          1/1     Running   0          68s
p06-company-5bc4d7fddd-wrkps            1/1     Running   0          68s
p06-event-96d7d97d9-mbqfg               1/1     Running   0          68s
p06-event-worker-76b49669c6-jplb7       1/1     Running   0          68s
p06-frontend-54bd9b878d-drdl2           1/1     Running   0          68s
p06-frontend-54bd9b878d-gkjzn           1/1     Running   0          68s
p06-gateway-568cc9c84c-6jts8            1/1     Running   0          67s
p06-gateway-568cc9c84c-znj5p            1/1     Running   0          67s
p06-job-5d7f9668bd-4hdcb                1/1     Running   0          67s
p06-job-5d7f9668bd-mwtj7                1/1     Running   0          67s
p06-job-worker-8466fff7f5-pnwr8         1/1     Running   0          67s
p06-kafka-0                             1/1     Running   0          67s
p06-mongo-0                             1/1     Running   0          67s
p06-student-69fcccf994-v995w            1/1     Running   0          66s
p06-student-69fcccf994-wq8kw            1/1     Running   0          67s
```

Two problems came up and were fixed (`RUN_LOG.txt` 6.2). `kind create cluster` failed at `kubeadm init`: Docker Desktop on this machine uses cgroup v1, and the kubelet of Kubernetes 1.35 and later, including kind's newest default image, refuses to start on it. The node image is now pinned to `kindest/node:v1.34.0` in `kind-config.yaml`. `kind load docker-image` failed with `content digest ... not found` because of the image store's attestation entries; `k8s/load_images.py` saves each image for one platform (`docker save --platform linux/amd64`) and loads the archive.

Not covered. No AWS or EKS here (section 7). The three Kafka failure demos of section 5 were run on Docker Compose, not inside Kubernetes. There was no load test and no autoscaler, and the website was fetched (HTTP 200) but not clicked through in a browser on the cluster.

5. Kafka flows

5.1 Deployment and topics

Kafka runs as one broker in KRaft mode: container `p06-kafka` in Docker Compose and the StatefulSet `p06-kafka` in Kubernetes. Inside both it is `p06-kafka:9092`; from this machine the Compose broker is

`localhost:9068`. Topics are named `p06.<event type>`, have 3 partitions and replication factor 1 (`p06.dlq` has 1), and are created by the first service or worker that needs them.

Topic	Producer	Consumer (group)	Key	What the consumer does
`p06.application.created`	Application API, `POST /jobs/{id}/apply`	Application worker (`p06-application-worker`)	student:job	saves the application (Pending) and an activity-log entry
`p06.application.updated`	Application API, `PUT /applications/{id}/resume`	Application worker	application id	points the application at the new resume, removes the old PDF if no other application uses it
`p06.application.status_changed`	Application API, `PATCH /applications/{id}/status`	Application worker	application id	writes a notification for the student
`p06.event.registration_created`	Event API, `POST /events/{id}/register`	Event worker (`p06-event-worker`)	student:event	writes a confirmation notification
`p06.job.created`	Job API, `POST /jobs`	Job worker (`p06-job-worker`)	job id	writes a notification for every student whose saved preferences match the job
`p06.dlq`	any worker	a person reads it	original key	messages that failed for good

The key decides the partition, so events with the same key reach the same worker in order. Only applications are written through Kafka: the other three flows save their data in the API first and use Kafka for what follows (notifications), so their endpoints answer as before.

5.2 The message

Every message is one JSON object in the Kafka value. Three of its fields are repeated in the Kafka headers, and the `trace_id` must be the same in both places or the message is rejected.

Field	Rule
`event_id`	a new UUID for every event; the key to idempotency
`event_type`	one of the five types above
`timestamp`	ISO 8601, UTC, ends in `Z`
`trace_id`	1 to 64 characters; in the Kafka header and in the body. It comes from the request that caused the event (`X-Trace-ID`, else the `X-Request-ID` the gateway adds), and everything that follows from the event carries the same value
`producer`	the service that sent it
`schema_version`	1
`payload`	the event's data, always a JSON object

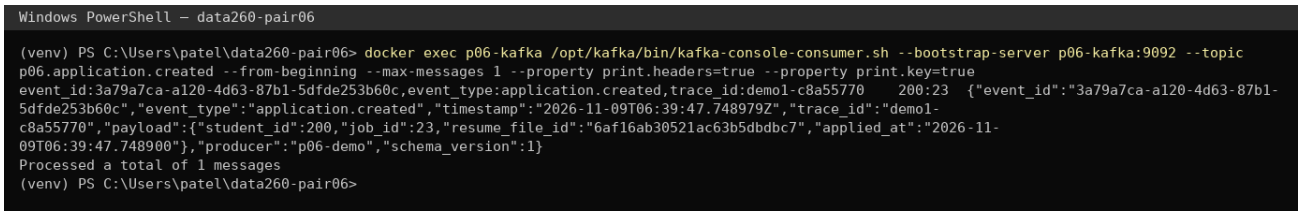


Figure 6: A real message read from p06.application.created with headers and key (sent by the demo script): headers event_id, event_type and trace_id, key 200:23, and the JSON body.

In the screenshot the key is `200:23` (student 200, job 23), the header `trace_id` and the body `trace_id` are both `demo1-c8a55770`, and the producer is `p06-demo` because this message was sent by the demo script of 5.5.

5.3 What makes the delivery safe

Kafka delivers at least once. The code makes the effect happen once.

Rule	How
The producer does not lose or duplicate a message	`acks=all` and an idempotent producer; the API waits up to 8 s for the broker's confirmation. If it cannot confirm, `POST /jobs/{id}/apply` answers 503 and removes the stored PDF.
A worker commits an offset only after its database write	a worker killed between the write and the commit gets the message again, and the handler turns the second run into a no-op
Handlers are idempotent	the application's natural key (student, job) has a unique index; notifications and activity-log entries written by a worker have an `_id` built from the `event_id`, so the same event cannot write them twice
Retries are bounded	a temporary failure is tried 3 times in all, pausing 1 s and then 2 s
A bad message does not block the others	a message that can never succeed (invalid JSON, unknown type, a student that does not exist) is copied to `p06.dlq` without retrying; one that still fails after the third attempt is copied there too; only then is its offset committed
A message is never silently dropped	if the copy to `p06.dlq` itself fails, the worker stops without committing and Docker or Kubernetes restarts it

One limit is known and documented (`docs/lab2/KAFKA_CONTRACT.md`, section 6): for status changes, registrations and new jobs the API saves first and publishes afterwards. If Kafka is down at that exact moment the data is saved but the notification is lost; the clean fix would be an outbox collection, which this lab does not build.

5.4 Tests and an end-to-end run

`tests/test_kafka.py` tests the message format, the handlers, the worker loop (with a fake consumer and producer), the publishers and the API events without a broker (59 tests when first written, more since). Two mutation checks showed that the tests notice if the key guarantees are broken (`RUN_LOG.txt` 4.3): with the duplicate check removed from the application handler, 4 tests failed; with the offset committed before the handler runs, 6 tests failed; with both files restored, all 59 passed.

An end-to-end script against the real broker (Compose, through the gateway, one `X-Trace-ID` per request) gave 18 passed, 0 failed (`RUN_LOG.txt` 4.5):

Flow	Result
Company posts a job	answered 201 at once; the Job worker created 27 notifications, exactly the students whose preferences match (computed independently from the `student_preferences` documents), each with the job's trace id
Student applies	202 Accepted in 31 ms; the application was saved by the worker, knows its `source_event_id` and `trace_id`; applying again is refused (409); exactly one application exists
Company changes the status	answered 200; the student got a notification from the worker 0.2 s later, carrying the company request's trace id
Student registers for an event	answered 201; the Event worker wrote a confirmation notification

5.5 The three failure demos

The demos are run by `backend/scripts/kafka_demos.py` against the real broker of the Docker Compose stack, and the script writes its own output to `docs/lab2/kafka_demos_output.txt`. All numbers below are from that file (run of 2026-11-09). Each demo uses a student and a job that have no application yet.

1. The same event twice, with the same key. The identical message (same `event_id`, key `200:23` and headers) was published twice.

What was observed	Result
Worker log	first message `event=processed`, second `event=duplicate` (same `event_id`)
Applications for student 200 and job 23	0 before, **1** after
Activity-log rows for that `event_id`	1
A different event (new `event_id`) for the same student and job, as after a double click	`event=duplicate`; still 1 application (unique index on student and job); the extra PDF was removed again

The consumer is idempotent: the database has no duplicate.

2. A worker killed during processing. To hit the narrow window between the database write and the offset commit, the worker has a small demo hook: if the file `/tmp/p06-crash-before-commit` exists, the worker deletes it and kills itself (`os._exit(137)`) right after the handler finished and before `consumer.commit()`. Without the file it does nothing. The demo arms the hook, publishes one event and watches.

What was observed	Result
Worker	died (container restart count 0 → 1; Docker restarted it)
Database right after the kill	**1** application: the write had happened, nothing was lost
Consumer group right after the kill	`p06.application.created`, partition 2: CURRENT-OFFSET 3, LOG-END-OFFSET 4, **LAG 1**: the commit never happened
Restarted worker	logged `event=started` at 06:39:50 and, 13 s later (the group rebalance waiting for the dead member), `event=duplicate` for the same `event_id`
After the restart	still **1** application; the same partition shows CURRENT-OFFSET 4, **LAG 0**

Was the event lost or reprocessed? It was **reprocessed**: Kafka still held the message because the offset had not been committed, delivered it again after the restart, and the idempotent handler found the application already there. Because the offset is committed only after the database write, a kill can only cause a repeat, never a loss; if the commit came first, the same kill would have lost the application (the mutation check in 5.4 covers this order). The script prints PASS only when the `duplicate` line for this `event_id` has appeared and the partition shows LAG 0, and FAIL after 60 s otherwise.

3. Poison messages, bounded retries and the dead-letter topic.

3a. Three messages that can never succeed were published, followed by a good one:

Message	Handling	Record on `p06.dlq`
text that is not JSON	no retries	`InvalidMessage`, attempts 0, `original_value` = the text
a valid envelope for student 999999, who does not exist	`PermanentError`, no retries	`PermanentError`, attempts 1, `student 999999 does not exist`
a header `trace_id` that differs from the body's	rejected, no retries	`InvalidMessage`, attempts 0, `header trace_id does not match the message body`
a good application right behind them	`event=processed`	the application was saved

The consumer group showed LAG 0 on every partition, and no application was created for student 999999: the bad messages were isolated and the pipeline continued.

3b. Bounded retries. MongoDB was paused (`docker pause p06-mongo`) while an application event was published:

Time	Worker log
06:40:44	`event=retry attempt=1/3 in=1.0s`
06:40:50	`event=retry attempt=2/3 in=2.0s`
06:40:57	`event=gave_up attempts=3`, then `event=dead_lettered`; record on `p06.dlq`: `ServerSelectionTimeoutError`, attempts 3

The application was not saved (0 for that pair), and a message holds the worker for about 13 s at most. After `docker unpause` the next message was `event=processed` and its application was saved. A message dead-lettered this way is not re-driven automatically; its `original_value` is on `p06.dlq` and can be published again by hand.

Mistakes made on the way, kept in `RUN_LOG.txt` section 5. The first version of demo 2 printed PASS before the restarted worker had reprocessed the message: it accepted any `duplicate` line, and demo 1 had just written one. The saved output showed LAG 1 still on that partition, and the `duplicate` line for the crashed event appeared only later, during demo 3a. The check now waits for the `duplicate` of that event's own `event_id` and LAG 0 (RUN_LOG 5.10). Earlier, the demo script published to the event type `application.created` instead of the topic `p06.application.created`; because automatic topic creation is off, the send was refused, which is the protection working. These demos ran on Docker Compose; in Kubernetes, Kafka and the workers were used and tested (section 4.4) but the three demos were not repeated there.

6. MongoDB schema and migration

6.1 Collections

The database is `p06_handshake` on MongoDB 7; every service reaches it through `backend/app/mongo.py` (the driver is `pymongo`). The full schema is `docs/lab2/MONGO_SCHEMA.md`.

Collection	Documents (seed)	Purpose	Written by
`students`	200	account and profile; skills and experience are inside the document	Student service
`companies`	50	company account and profile	Company service
`jobs`	300	job postings; skills inside	Job service
`applications`	1,500	one application per student and job	Application worker (status changes: Application service)
`events`	100	events; eligible majors inside	Event service
`event_registrations`	400	one registration per student and event	Event service
`student_preferences`	151	saved job and event preferences, one per student	Student service
`saved_items`	150	jobs and events a student saved	the assistant, after the student confirms
`sessions`	0	one document per login; expires by itself	login and signup
`activity_logs`	0	audit trail of what users did	every service and the Application worker
`notifications`	0	short messages written by the Kafka workers	the three workers
`counters`	9	the next integer id per collection	`mongo.next_id`
`resumes.files`, `resumes.chunks`	1	GridFS: private resume PDFs (the seed stores one placeholder PDF that all seeded applications share)	Application service
`profile_pics.files`, `profile_pics.chunks`	0	GridFS: public pictures and logos	Student and Company services

The Lab 1 MySQL tables map as follows: `students`, `student_skills` and `student_experience` became `students`; `jobs` and `job_skills` became `jobs`; `events` and `event_eligible_majors` became `events`; the other tables kept their names. `sessions`, `activity_logs`, `counters` and `notifications` are new, and the files moved from the `uploads/` folder into GridFS.

Conventions that make the migration invisible to the website and the assistant:

- **Integer ids.** A `counters` collection hands out the next whole number per collection (`find_one_and_update` with `$inc`), so URLs such as `/jobs/12` and the assistant tools do not change.
- **Dates.** A date without a time is stored as text (`"2026-10-05"`); a date-time is a real MongoDB date. The business timestamps are the server's clock (UTC in the containers); sessions and activity logs are always UTC.
- **Embedding.** Things that never exist without their parent are inside it (a student's skills and experience); things that are looked up on their own (applications, registrations, saved items, preferences) are collections.
- **Derived search fields.** `name_lower`, `skills_lower`, `salary_yearly_max` and similar are written together with the field they copy; documents are built only through `backend/app/documents.py`.

- **Files.** Resumes and pictures are in GridFS, so no container needs a disk.

6.2 Indexes

All indexes are one list (`INDEXES` in `mongo.py`), created by the seed script and by `get_db()` on the first request that finds MongoDB. A violated unique index raises `DuplicateKeyError`, which the API turns into 409.

Collection	Index	Kind	Why
`students`, `companies`	`email`	unique	one account per email
`applications`	`(student_id, job_id)`	unique	one application per student and job; stops a double click and makes the worker idempotent
`event_registrations`	`(student_id, event_id)`	unique	one registration per student and event
`saved_items`	`(student_id, item_type, item_id)`	unique	the same item cannot be saved twice
`student_preferences`	`student_id`	unique	one preferences document per student
`sessions`	`expires_at`	TTL (`expireAfterSeconds: 0`)	MongoDB deletes expired sessions
`sessions`	`(user_id, role)`	normal	find a user's sessions
`jobs`	`company_id`, `deadline`, `(posting_date desc, _id desc)`, `category`, `skills_lower`, `salary_yearly_max desc`	normal	search, "my postings", sorting
`applications`	`job_id`; `(student_id, applied_at desc)`	normal	applicants of a job; my applications; the activity heatmap
`students`, `events`, `event_registrations`, `activity_logs`, `notifications`	name, major, skills, date, event and user fields (see `MONGO_SCHEMA.md` section 4)	normal	directory filters, event search, a user's history and notifications

6.3 Security: bcrypt and sessions with expiry

- **Passwords** are hashed with bcrypt (12 rounds) before they are stored. No endpoint returns `password_hash`. The 67 live checks confirm that all stored passwords are bcrypt hashes and that a seeded student and a seeded company can log in with them.
- **Sessions.** Every signup and login creates a document in `sessions` valid for 8 hours. The JWT (HS256) carries its id as the `jti` claim and is accepted only while that session exists, so `POST /auth/logout` deletes the session and the token stops working at once. The TTL index makes MongoDB delete expired sessions itself; MongoDB checks about once a minute, so the API also compares `expires_at` with the clock on every request. The live checks showed that the same token is refused after logout, that MongoDB deleted

an expired session by itself (after waiting for its TTL monitor) and that a session that had not expired was kept.

- **Role views and private files.** A company sees a student's email, phone and GPA but never the date of birth; a student sees peers without email, phone, GPA, date of birth or city. Resumes are served only by `GET /applications/{id}/resume` after checking that the caller is the applicant or the company that owns the job; anyone else gets 404, so the existence of the resume is not revealed. Profile pictures are public by design.
- **Secrets.** The connection string, which can contain the MongoDB password, comes only from `MONGO_URL` (a Compose `.env` value or a Kubernetes Secret). It is never in the code or the images. In Compose and Kubernetes MongoDB runs with a user and password.

6.4 Evidence that the migration did not change the platform

- **Same API, old code against new code.** The Lab 1 code (tag `lab1`, running on MySQL) and the new code (MongoDB) were run side by side and 382 requests were compared: job filter combinations, pages, ids, application and event filters, student directory filters, the company views, heat-map windows and 45 resume downloads. Of the 382, 374 gave identical JSON. The other 8 were explained, not assumed: they concern the demo student's data in the old database (an extra skill and new experience ids added during the Lab 1 demos), the time of the seed run, and the date of birth that Faker computes from today's date, none of them the migration (`RUN_LOG.txt` 2.4). One real difference was found and fixed on the way: the `Content-Disposition` header of the resume download.
- **67 live checks** (counts, indexes, bcrypt, sessions and logout, GridFS pictures and resumes, role privacy, the job, application and event workflows, the activity log, MongoDB's own session expiry): 67 passed, 0 failed, first against the API on this laptop, then through the gateway to the six service containers, then against the Compose stack with a password on MongoDB (`RUN_LOG.txt` 2.5 and 3.8).
- **Bugs found by the tests and fixed.** The first full run after the rewrite gave 4 failed, 404 passed and 26 errors: updating a student's profile dropped the unset fields inside each experience entry, so stored entries lacked `end_date` and `description`; `documents.experience()` now always writes every key. A mutation check removed the unique flag of the applications index: the database-level test failed, while the API-level tests still passed because the API checks for a duplicate before inserting, which is why the index is tested on its own.
- **Backend tests:** 405 (Lab 1) → 461 after the migration → 543 now, all passing; the frontend suite has 211 tests (`RUN_LOG.txt` 4.3).

6.5 Evidence used for sections 1 to 6

Source	Used for
`RUN_LOG.txt` (Lab 2, phases 1 to 6)	commands and real output: builds, health checks, gateway routing, the MongoDB checks, Compose, Kafka tests, the Kubernetes test, mistakes and fixes
`docs/lab2/kafka_demos_output.txt`	the numbers of the three Kafka failure demos
`docs/lab2/screenshots/compose_ps.png`, `kafka_message.png`, `assistant_dependencies.png`	figures 3, 4 and 6
`docs/lab2/KAFKA_CONTRACT.md`, `MONGO_SCHEMA.md`, `ARCHITECTURE.md`	topics, message format, schema, indexes, diagrams
`k8s/` and `docker-compose.yml`, `backend/docker/`, `gateway/`, `frontend/Dockerfile`	the manifests, images and Compose configuration described above

Partner sections (placeholders)

The sections below belong to Sreeramachandra Sai Achutuni. Each box names what the assignment asks for so that nothing is forgotten; the content, screenshots and numbers must come from the partner's own runs.

7. Cloud deployment (AWS ECR and EKS)

PLACEHOLDER (partner): AWS ECR and EKS deployment. To add: the ECR repositories and pushed images with the `p06-` names, the EKS cluster, the `p06-handshake` namespace, running pods, Kubernetes Services, the LoadBalancer or Ingress endpoint, one successful end-to-end request with request and response, `kubectl` output, and how the services communicate and scale. Notes for this work are at the end of `k8s/README.md` (images from ECR, LoadBalancer Services, the website image built with the load balancer address, the EBS storage driver, an Ollama address). Hide access keys, passwords and tokens in every screenshot.

8. Memory and consolidation

PLACEHOLDER (partner): three-layer assistant memory. To add: `assistant_rules`, `student_memory` and session conversations; memory injected into the prompt and the `/memory` command; consolidation as a Kafka consumer (the conventions are in `docs/lab2/KAFKA_CONTRACT.md` section 5 and `docs/lab2/MONGO_SCHEMA.md`); a student memory with at least 60 entries (three contradictions, three duplicates), the memory before and after consolidation (under 40 entries, duplicates merged, contradictions resolved), and one consolidation decision explained, including any case the model handled incorrectly.

9. Redux state

PLACEHOLDER (partner): Redux. To add: the required slices, actions and selectors, and the Redux DevTools evidence (including how the applications slice handles the 202 answer of `POST /jobs/{id}/apply`).

10. Performance (JMeter)

PLACEHOLDER (partner): JMeter plan, CRUD load tests and performance graphs.

11. Assistant load analysis

PLACEHOLDER (partner): assistant load tests and their analysis.

12. Token compaction

PLACEHOLDER (partner): token tracking, compaction and recall evidence.

13. One end-to-end trace

PLACEHOLDER (partner): one trace followed through the system. To add: the `trace_id` across the API, the Kafka headers and body, the worker log and the assistant stages. The Kafka part of the trace is described in section 5.2 and the apply flow in section 2.

14. Guardrails and the evaluation harness

PLACEHOLDER (partner): agent guardrail results, the harness test matrix and output, reliability before and after repair, and the four cases: one blocked injection, one declined confirmation, one repaired malformed call and one bounded stop. The harness output files (`agent_harness.json` or `.jsonl`) and the human-readable summary belong here.